

Test case catalog — Rethink Clinical (Phase 1)

Printable copy: [test-cases.pdf](#) .

Catalog of **automated** test cases already implemented against `clinical.dev2` . Each case follows a standard test-case structure (ID, objective, type, preconditions, steps, expected result). Traceability IDs match [coverage-matrix.md](#) . Defects live in [defects.md](#) .

How to read status

Status	Meaning
Pass	Included in <code>npm test</code> ; last consecutive green runs on 2026-08-27
Known fail (@bug)	Asserts <i>correct</i> product behaviour the app does not have yet. Excluded from <code>npm test</code> . Run with <code>npx cucumber-js -p bugs</code> .

Shared preconditions (unless a case says otherwise)

1. `BASE_URL` and `API_BASE_URL` are reachable (preflight).
2. The suite has harvested a session from `/temp-dev-login` (no dedicated test account required).
3. A **resolved client** exists: an active client with at least one program that has at least one target. IDs are not hardcoded.
4. Each UI case starts in a **fresh browser context** seeded with auth only (no leftover demo-session or saved-report state).
5. Counts are asserted as **relationships** (UI vs API), not as fixed numbers.

Execution

Profile	Command
Default (Pass cases)	<code>npm test</code>
Smoke (P0 UI/API)	<code>npm run test:smoke</code>
Known defects	<code>npx cucumber-js -p bugs</code>

Index

Module	IDs	Count	Default run
Foundations	FND-1 ... FND-7	6 scenarios + harness	Yes
Authentication	AUTH-1 ... AUTH-5	5	Yes
Clients	CLI-1 ... CLI-9	10	Yes
Programs	PRG-1 ... PRG-8	12	Yes
Analyze Data	AZ-1 ... AZ-9	17 (2 of which @bug)	15 Pass + 2 Known fail
Behavior Support	BS-1 ... BS-3	3	1 Pass + 2 Known fail
Negative	NEG-1 ... NEG-5	5	Yes

Total executable		58	54 Pass + 4 Known fail
------------------	--	----	------------------------

Outline examples are listed as separate cases (AZ-5a–e, AZ-8a–c, PRG-3a-targets / PRG-3a-objectives).

0. Foundations

Feature: features/preflight/foundations.feature

Tags: @preflight

FND-2 — Both origins answer before any test runs

Field	Detail
Objective	Fail fast if the app or API gateway is unreachable, with an actionable message.
Type	Harness / API
Priority	P0
Preconditions	Environment variables for BASE_URL and API_BASE_URL are set.
Test data	None.
Steps	1. Before the suite starts, GET the application origin. 2. GET the API gateway origin.
Expected result	Application origin returns HTTP 200. API origin answers (unauthenticated 401 is acceptable). If either is down, the suite aborts with a message naming the origin.
Automation	foundations.feature — <i>Both origins answer before any test runs</i>
Status	Pass

FND-1 — A session is harvested from the preview sign-in

Field	Detail
Objective	Obtain a usable bearer session without storing credentials (D1).
Type	UI → token harvest
Priority	P0
Preconditions	/temp-dev-login is available on the app origin.
Test data	Session is read from localStorage["bh_clinical_auth_session"].
Steps	1. Drive /temp-dev-login in a throwaway browser context. 2. Read access token and expiry from localStorage. 3. Call a protected API with that token.
Expected result	Access token is present and not expired. API accepts the token (HTTP 200 on a protected call).
Automation	<i>A session is harvested from the preview sign-in</i>
Status	Pass

FND-3 — A test client is resolved by capability rather than by id

Field	Detail
Objective	Pick fixture data by capability so tests survive client-id churn (D2, D6).
Type	API
Priority	P0
Preconditions	Harvested token. At least one active client has a program with targets.
Test data	Resolver walks clients and programs; optional <code>TEST_CLIENT_ID</code> only <i>narrows</i> candidates.
Steps	1. Request clients. 2. For candidates, request programs then targets. 3. Select the first client with a program that has ≥ 1 target (preferring an active program).
Expected result	Resolved client is active. Resolved program has at least one target.
Automation	<i>A test client is resolved by capability rather than by id</i>
Status	Pass

FND-5 — A seeded context is authenticated and free of prior state

Field	Detail
Objective	Prove scenario isolation: auth is injected, demo data-collection state is not.
Type	UI
Priority	P0
Preconditions	Fresh context seeded with the harvested session only.
Steps	1. Open <code>/clients</code> . 2. Inspect the app shell and localStorage.
Expected result	Shell shows a signed-in user. Browser holds no <code>clinical.rbt.session.demo-session-*</code> keys.
Automation	<i>A seeded context is authenticated and free of prior state</i>
Status	Pass

FND-5b — An unseeded context is not authenticated

Field	Detail
Objective	Confirm the harness does not leak auth into a <code>@signed-out</code> scenario.
Type	UI
Priority	P0
Preconditions	Browser context is not seeded with a session (<code>@signed-out</code>).
Steps	1. Open the application root.

Expected result	Sign-in page is shown.
Automation	<i>An unseeded context is not authenticated</i>
Status	Pass

FND-7 — Credential fields are stripped before the browser sees them

Field	Detail
Objective	staff-role must not expose apiKey / password (and related fields) to the page or traces (D7).
Type	UI + network intercept
Priority	P0
Preconditions	Response scrubbing installed on the browser context.
Steps	1. Open the clients page. 2. Capture the staff-role response as the browser received it.
Expected result	Captured JSON has no populated credential fields. App shell still shows the signed-in user (scrubbing must not break the UI).
Automation	<i>Credential fields are stripped before the browser sees them.</i> Also <code>npm run verify:scrub</code> .
Status	Pass

FND-4 (page objects) and FND-6 (@wip gating, report script) are **harness work**, not Gherkin cases.

1. Authentication

Feature: features/auth/authentication.feature

Tags: @auth

AUTH-1 — The issued session is well formed

Field	Detail
Objective	The harvested session has the token pair and expiry contract the app uses.
Type	API / session
Priority	P0 (smoke)
Preconditions	Session harvested in BeforeAll.
Steps	1. Inspect harvested session fields.
Expected result	Access token and refresh token are present. Access expiry is in the future. Refresh expiry is on or after access expiry.
Automation	<i>The issued session is well formed</i> — @api @smoke @auth
Status	Pass

AUTH-2 — A refresh token exchanges for a new session

Field	Detail
Objective	Refresh-token exchange returns a new, usable access token (D8: own sign-in, not the shared token).
Type	API
Priority	P1
Preconditions	A fresh /temp-dev-login session for this case only (refresh tokens are single-use).
Steps	1. Sign in independently. 2. POST refresh with that refresh token and x-application-key. 3. Call staff-role with the new access token.
Expected result	HTTP success with access token, refresh token, and both expiries. New access token ≠ previous. staff-role returns 200.
Automation	<i>A refresh token exchanges for a new session</i> — @api @auth
Status	Pass

AUTH-3 — The preview sign-in lands an unauthenticated visitor on the clients page

Field	Detail
Objective	/temp-dev-login authenticates a visitor and lands them on the clients list.
Type	UI
Priority	P0 (smoke)
Preconditions	@signed-out — no seeded session.
Steps	1. Open /temp-dev-login.
Expected result	User arrives on the clients page. App shell shows a signed-in user.
Automation	<i>The preview sign-in lands an unauthenticated visitor on the clients page</i> — @ui @smoke @auth
Status	Pass

AUTH-4 — Staff role identifies the current user

Field	Detail
Objective	staff-role returns the signed-in user's role identity.
Type	API
Priority	P1
Preconditions	Harvested access token.
Steps	1. GET staff-role.

Expected result	HTTP 200. Body names a role and a user.
Automation	<i>Staff role identifies the current user</i> — @api @auth
Status	Pass

AUTH-5 — A protected client record is closed to an unauthenticated visitor

Field	Detail
Objective	Deep-linking a client Analyze Data URL without a session must not show clinical content.
Type	UI (negative)
Priority	P1
Preconditions	@signed-out. A resolved client id is known to the suite.
Steps	1. Open /clients/{id}/analyze-data without auth.
Expected result	Sign-in page is shown. No client-record content is rendered.
Automation	<i>A protected client record is closed to an unauthenticated visitor</i> — @ui @negative @auth
Status	Pass

2. Clients list

Feature: features/clients/clients.feature

Tags: @clients

CLI-1 — The clients endpoint returns a coherent paged envelope

Field	Detail
Objective	Clients list API is a valid paged envelope (no fixed totalCount).
Type	API
Priority	P0 (smoke)
Steps	1. GET /clinical/v1/clients.
Expected result	HTTP 200. Content-Type includes x-api-version=1. Body has page, pageSize, totalCount, totalPages, items. totalPages = max(1, ceil(totalCount / pageSize)). Item length ≤ pageSize; if one page, items.length = totalCount.
Automation	<i>The clients endpoint returns a coherent paged envelope</i>
Status	Pass

CLI-2 — Every client item carries the fields the app depends on

Field	Detail
-------	--------

Objective	Each client row the UI needs is present on every item.
Type	API
Priority	P1
Steps	1. GET clients list.
Expected result	Every item has a numeric <code>id</code> , a name, a client number, and an active flag.
Automation	<i>Every client item carries the fields the app depends on</i>
Status	Pass

CLI-3 — The clients page lists exactly the clients the API returned

Field	Detail
Objective	UI list membership matches the API (set of ids), not a hardcoded count.
Type	UI
Priority	P0 (smoke)
Steps	1. Open <code>/clients</code> . 2. Compare listed client ids with the API <code>items</code> ids.
Expected result	The two sets of ids are equal.
Automation	<i>The clients page lists exactly the clients the API returned</i>
Status	Pass

CLI-4 — Opening a client lands on their Skills Programs workspace

Field	Detail
Objective	Selecting a client from the list opens that client's workspace.
Type	UI
Priority	P0
Steps	1. Open <code>/clients</code> . 2. Open the resolved client from the table.
Expected result	Workspace is shown for that client. Client switcher names that client.
Automation	<i>Opening a client lands on their Skills Programs workspace</i>
Status	Pass

CLI-5 — Searching by name narrows the list to matching clients

Field	Detail
Objective	Name filter is client-side and inclusive of the resolved client.
Type	UI

Priority	P1
Test data	A substring of the resolved client's name.
Steps	1. Open <code>/clients</code> . 2. Type a partial name into the name search.
Expected result	Every remaining row's name contains the search text. The resolved client is still listed.
Automation	<i>Searching by name narrows the list to matching clients</i>
Status	Pass

CLI-5b — Searching by an unmatched name empties the list

Field	Detail
Objective	A name that matches nobody yields an empty table (not an error).
Type	UI
Priority	P1
Test data	zzz-no-such-client
Steps	1. Open <code>/clients</code> . 2. Search by that name.
Expected result	No client rows are listed.
Automation	<i>Searching by an unmatched name empties the list</i>
Status	Pass

CLI-6 — Searching by client ID narrows the list to one client

Field	Detail
Objective	Client-number search isolates the resolved client.
Type	UI
Priority	P1
Test data	Resolved client's client number.
Steps	1. Open <code>/clients</code> . 2. Search by that client number.
Expected result	Only the resolved client is listed.
Automation	<i>Searching by client ID narrows the list to one client</i>
Status	Pass

CLI-7 — The client switcher moves to the chosen client's record

Field	Detail
Objective	Header client switcher navigates to the selected client.

Type	UI
Priority	P1
Steps	1. Open <code>/clients</code> . 2. Choose the resolved client in the PrimeNG client switcher.
Expected result	Workspace URL/state is that client's record.
Automation	<i>The client switcher moves to the chosen client's record</i>
Status	Pass

CLI-8 — The clients page fires the expected calls and no failures

Field	Detail
Objective	Page load issues the expected first-party calls without HTTP failures.
Type	Network
Priority	P2
Steps	1. Attach a network recorder. 2. Open <code>/clients</code> .
Expected result	Runtime config, staff-role, and clients calls succeeded. No recorded first-party call failed.
Automation	<i>The clients page fires the expected calls and no failures — @network</i>
Status	Pass

CLI-9 — Each row's status agrees with the API active flag

Field	Detail
Objective	Visible status column matches API active / equivalent for each id.
Type	UI
Priority	P2
Steps	1. Open <code>/clients</code> . 2. For each row, compare UI status with API active flag.
Expected result	Every row agrees with the API for that client.
Automation	<i>Each row's status agrees with the API active flag</i>
Status	Pass

3. Client programs

Feature: `features/programs/programs.feature`

Tags: `@programs`

PRG-1 — The programs endpoint describes a client's programs

Field	Detail
Objective	Programs list for the resolved client is a valid envelope with required fields.
Type	API
Priority	P1
Steps	1. GET /clinical/v1/clients/{id}/programs.
Expected result	HTTP 200. Paging arithmetic holds. Every item has numeric id, non-empty title, boolean active.
Automation	<i>The programs endpoint describes a client's programs</i>
Status	Pass

PRG-2 — The program library returns the template catalogue

Field	Detail
Objective	Program library is a paged catalogue of templates.
Type	API
Priority	P2
Steps	1. GET /clinical/v1/program-library.
Expected result	HTTP 200. Paging holds. Every entry has id and non-empty title. Library is not empty.
Automation	<i>The program library returns the template catalogue</i>
Status	Pass

PRG-3a-targets — Targets return a valid paged envelope

Field	Detail
Objective	Per-program targets uses the paged envelope (not a bare array).
Type	API
Priority	P1
Steps	1. GET .../programs/{programId}/targets.
Expected result	HTTP 200. Paging arithmetic is self-consistent.
Automation	Outline <i>Paged per-program endpoints</i> — targets
Status	Pass

PRG-3a-objectives — Objectives return a valid paged envelope

Field	Detail
-------	--------

Objective	Per-program objectives uses the paged envelope.
Type	API
Priority	P1
Steps	1. GET .../programs/{programId}/objectives.
Expected result	HTTP 200. Paging arithmetic is self-consistent.
Automation	Outline — objectives
Status	Pass

PRG-3b — Mastery criteria returns a programme-scoped criteria document

Field	Detail
Objective	mastery-criteria is { programId, phases[] }, not a paged envelope.
Type	API
Priority	P1
Steps	1. GET .../mastery-criteria.
Expected result	HTTP 200. programId equals the requested program. phases is an array.
Automation	<i>Mastery criteria returns a programme-scoped criteria document</i>
Status	Pass

PRG-3c — Target groups returns a bare list

Field	Detail
Objective	target-groups is a JSON array (may be empty).
Type	API
Priority	P1
Steps	1. GET .../target-groups.
Expected result	HTTP 200. Body is an array.
Automation	<i>Target groups returns a bare list</i>
Status	Pass

PRG-3d — Data collection describes how the programme is measured

Field	Detail
Objective	data-collection names the program and a collection method.
Type	API

Priority	P1
Steps	1. GET .../data-collection.
Expected result	HTTP 200. programId matches. method is non-empty. prompts is an array.
Automation	<i>Data collection describes how the programme is measured</i>
Status	Pass

PRG-4 — Flagged automastery evaluations return only flagged items

Field	Detail
Objective	automastery-evaluations?status=flagged is not a vacuous empty pass.
Type	API
Priority	P1
Preconditions	At least one of the resolved client's programs has flagged evaluations (else the case errors out rather than passing on []).
Steps	1. Walk programs until one has flagged items. 2. Assert that response.
Expected result	HTTP 200. Every item has status = flagged and a numeric targetId. List is non-empty.
Automation	<i>Flagged automastery evaluations return only flagged items</i>
Status	Pass

PRG-5 — The rail's Current tab lists exactly the active programs

Field	Detail
Objective	Current tab = programs with active: true (not the full API list).
Type	UI
Priority	P1
Steps	1. Open /clients/{id} (Skills Programs). 2. Read rail item ids. 3. Re-read programs API and compare (D9).
Expected result	Rail ids equal the live API's active program ids.
Automation	<i>The rail's Current tab lists exactly the active programs</i>
Status	Pass

PRG-7 — Current and Inactive tabs partition the client's programs

Field	Detail
-------	--------

Objective	The two tabs are disjoint and together cover every program.
Type	UI
Priority	P2
Steps	1. Open workspace. 2. Record Current ids. 3. Switch to Inactive and record ids. 4. Compare with a fresh programs API read.
Expected result	No id on both tabs. Every API program id appears on one tab.
Automation	<i>Current and Inactive tabs together list every program exactly once</i>
Status	Pass

PRG-6 — Selecting a program reveals its details

Field	Detail
Objective	Selecting the resolved program shows details panels.
Type	UI
Priority	P1
Steps	1. Open workspace. 2. Switch to Current or Inactive as needed. 3. Click the resolved program.
Expected result	Targets, goals, and settings panels (program-details-*) are visible.
Automation	<i>Selecting a program reveals its details</i>
Status	Pass

PRG-8 — The domain filter narrows the rail to one domain

Field	Detail
Objective	Domain filter restricts Current-tab programs; "All domains" restores the active set.
Type	UI
Priority	P2
Test data	Most common domain among currently active programs (live API).
Steps	1. Open workspace. 2. Filter by that domain. 3. Assert rail vs API. 4. Select "All domains". 5. Assert full active set.
Expected result	After filter, rail = active programs in that domain. After clear, rail = all active programs.
Automation	<i>The domain filter narrows the rail to one domain</i>

Status	Pass
--------	------

4. Analyze Data

Feature: features/analyze-data/analyze-data.feature

Tags: @analyze-data

AZ-1 — The three summary tiles render numbers

Field	Detail
Objective	Mastered, in-scope, and remaining tiles show non-negative numbers (not --).
Type	UI
Priority	P1
Steps	1. Open /clients/{id}/analyze-data. 2. Wait until in-scope shows a digit. 3. Read all three tiles.
Expected result	Each tile is visible and contains a finite number ≥ 0 .
Automation	<i>The three summary tiles render numbers</i>
Status	Pass

AZ-2 — Targets in scope reconciles with the targets API

Field	Detail
Objective	In-scope equals the sum of totalCount across all programs' targets APIs; identity mastered + remaining = in scope.
Type	UI + API
Priority	P1
Steps	1. Open Analyze Data. 2. Poll until tile in-scope equals live target total. 3. Check mastered + remaining.
Expected result	In-scope = sum of targets over every program. Arithmetic identity holds.
Automation	<i>Targets in scope reconciles with the targets API</i>
Status	Pass

AZ-3 — The skill-area chart plots one category per domain

Field	Detail
Objective	Highcharts x-axis categories are the client's distinct program domains; summary "across N skill areas" matches.

Type	UI + API
Priority	P2
Steps	1. Open Analyze Data. 2. Read chart SVG y-axis ticks and x-axis labels. 3. Compare with distinct domain values from programs API.
Expected result	Chart SVG visible with y-axis ticks. Categories equal distinct domains. Skill-area count = category count.
Automation	<i>The skill-area chart plots one category per domain</i>
Status	Pass

AZ-4 — Pending mastery determinations are grouped by program

Field	Detail
Objective	Review rows match flagged automastery evaluations; grouping headings match each row's program.
Type	UI + API
Priority	P2
Steps	1. Open Analyze Data. 2. Count mastery-review-row. 3. Sum flagged evaluations across programs. 4. Check each group heading vs row program names.
Expected result	Row count = flagged total. Headings unique. Every row's program name matches its group heading (case-insensitive).
Automation	<i>Pending mastery determinations are grouped by program</i>
Status	Pass

AZ-5a ... AZ-5e — Date range chip is the sole active selection

Field	Detail
Objective	Each date-window chip becomes the only aria-pressed="true" chip.
Type	UI
Priority	P2
Test data	One case per window: 2 weeks, 1 month, 3 months, 6 months, All.
Steps	1. Open Analyze Data. 2. Click the chip.
Expected result	Only that window is active.
Automation	Outline <i>Each date range chip becomes the sole active selection</i>
Status	Pass (5 cases)

AZ-6 — Loading the report fetches targets for every program

Field	Detail
Objective	Page load fans out a targets request per program and at least one automastery call.
Type	Network
Priority	P2
Steps	1. Record traffic. 2. Open Analyze Data. 3. Wait for network idle.
Expected result	A targets URL exists for every program id known at assertion time. ≥ 1 automastery-evaluations request.
Automation	<i>Loading the report fetches targets for every program</i>
Status	Pass

AZ-6b — Every request the report makes succeeds

Field	Detail
Objective	Per-program targets and automastery-evaluations calls return HTTP status below 400.
Type	Network
Priority	P2
Defect	DEF-4 — intermittent 500 under concurrent fan-out.
Steps	Same as AZ-6.
Expected result	No per-program request has status ≥ 400 .
Automation	<i>Every request the report makes succeeds</i> — @bug
Status	Known fail (@bug)

AZ-7 — Changing the grouping regroups the chart

Field	Detail
Objective	Selecting Category regroups the x-axis away from Domain.
Type	UI + API
Priority	P2
Defect	DEF-1 — select label changes; chart stays on Domain.
Steps	1. Open Analyze Data. 2. Snapshot domain categories. 3. Group by Category.
Expected result	Chart categories cover distinct program category values and differ from the domain snapshot.
Automation	<i>Changing the grouping regroups the chart</i> — @bug
Status	Known fail (@bug)

AZ-7b — The grouping select offers domain, category and area

Field	Detail
Objective	The grouping control at least exposes the three options.
Type	UI
Priority	P2
Steps	1. Open Analyze Data. 2. Open grouping select.
Expected result	Options Domain , Category , and Area are visible.
Automation	<i>The grouping select offers domain, category and area</i>
Status	Pass

AZ-8a ... AZ-8c — Mode tabs switch the report view

Field	Detail
Objective	Mode tabs are exclusive; the matching panel is shown and the others are absent.
Type	UI
Priority	P2
Test data	custom → custom panel; bulk → bulk panel; mastered → mastered panel.
Steps	1. Open Analyze Data. 2. Click the mode.
Expected result	Only that mode is aria-pressed. Named panel visible; other two report panels have count 0.
Automation	Outline <i>Mode tabs switch the report view</i>
Status	Pass (3 cases)

AZ-9 — Custom Graph offers every program as a comparison series

Field	Detail
Objective	"N available in this scope" equals the programs API item count.
Type	UI + API
Priority	P2
Steps	1. Open Analyze Data. 2. Switch to custom mode. 3. Compare series count with live programs list.
Expected result	Series count = program count.
Automation	<i>Custom Graph offers every program as a comparison series</i>
Status	Pass

5. Behavior Support

Feature: features/behavior-support/behavior-support.feature

Tags: @behavior-support

BS-1 — The page renders its plan rail and novel behaviors panel

Field	Detail
Objective	Chrome that is true whether or not behaviorplans is healthy.
Type	UI
Priority	P1
Steps	1. Open /clients/{id}/behavior-support. 2. Wait until the app stops retrying behaviorplans.
Expected result	Plan rail visible. Current and Inactive tabs match Current (n) / Inactive (n). Novel behaviors panel shows a numeric count.
Automation	<i>The page renders its plan rail and novel behaviors panel</i>
Status	Pass

BS-2 — Behavior plans returns the client's plans

Field	Detail
Objective	Observations behaviorplans should succeed for the resolved client.
Type	API
Priority	P1
Defect	DEF-2 — HTTP 500 for all clients.
Steps	1. GET /observations/v1/client/{id}/behaviorplans.
Expected result	HTTP 200.
Automation	<i>Behavior plans returns the client's plans</i> — @bug
Status	Known fail (@bug)

BS-3 — The page does not claim there are no plans while data is unavailable

Field	Detail
Objective	Empty-state copy and unavailable notice must not both be shown (DEF-5).
Type	UI
Priority	P1
Defect	DEF-5 — both messages visible after the retry fails.

Steps	1. Open Behavior Support and wait until behaviorplans traffic goes quiet. 2. Check visibility of empty rail message and unavailable notice.
Expected result	Not both visible.
Automation	<i>The page does not claim there are no plans while data is unavailable — @bug</i>
Status	Known fail (@bug)

6. Negative and error cases

Feature: features/negative/negative.feature

Tags: @negative

NEG-1 — An API call without a token is rejected

Field	Detail
Objective	Unauthenticated clients list is rejected.
Type	API
Priority	P1
Preconditions	No Authorization header.
Steps	1. GET /clinical/v1/clients.
Expected result	HTTP 401.
Automation	<i>An API call without a token is rejected</i>
Status	Pass

NEG-2 — An API call with a malformed token is rejected

Field	Detail
Objective	Garbage bearer token is rejected.
Type	API
Priority	P2
Test data	Authorization: Bearer not-a-real-token
Steps	1. GET /clinical/v1/clients.
Expected result	HTTP 401.
Automation	<i>An API call with a malformed token is rejected</i>
Status	Pass

NEG-5 — An unknown client id returns an empty list rather than an error

Field	Detail
Objective	Document current API behaviour for a non-existent client (not 404).
Type	API
Priority	P2
Test data	<code>max(existing client ids) + 1</code>
Steps	1. Derive a missing id. 2. GET that client's programs.
Expected result	HTTP 200. <code>items</code> is empty.
Automation	<i>An unknown client id returns an empty list rather than an error</i>
Status	Pass

NEG-3 — An unknown client id in the URL falls back to the clients list

Field	Detail
Objective	Deep link to a non-existent client does not crash or blank the shell.
Type	UI
Priority	P2
Test data	Same derived missing id as NEG-5.
Steps	1. Navigate to <code>/clients/{missingId}</code> .
Expected result	Clients list page is shown. URL settles on <code>/clients</code> .
Automation	<i>An unknown client id in the URL falls back to the clients list</i>
Status	Pass

NEG-4 — An unknown route falls back to the clients list

Field	Detail
Objective	Unknown path is handled without a blank app (product has no dedicated 404 page).
Type	UI
Priority	P2
Test data	<code>/no-such-route-xyz</code>
Steps	1. Open that path while authenticated.
Expected result	Clients list is shown. URL is <code>/clients</code> .
Automation	<i>An unknown route falls back to the clients list</i>

Status	Pass
--------	------

Out of scope (not in this catalog)

These are **not** covered as test cases yet:

- Write flows: add target, confirm/dismiss mastery, save report, record data (decision D4).
- Nav areas never crawled: Staff, Supervision, Settings, Training, Reporting, Template, Schedule, Notifications, Billing.
- CI pipeline (decision D5).